

Database System Composite Experiment Report

Online Store Database Management System

Name: REJEBOV ARSLAN

Student ID: 312024704046

Course: Database System & its Application

Submit Date: 7 July 2026

Part II – Project Introduction

2.1 Project Overview

The **Online Store Database Management System** is a relational database application designed to support the daily operations of an online retail business. The system provides an efficient and reliable way to manage products, customers, orders, payments, suppliers, and inventory. By storing all business information in a centralized MySQL database, the system improves data accuracy, reduces redundancy, and enables fast access to information for different users.

The database is designed according to database design principles, including entity-relationship modeling, normalization, primary and foreign key constraints, and indexing. These techniques ensure data integrity, consistency, and efficient query performance. The system also makes use of SQL views and triggers to simplify data access and automate business processes such as inventory updates and member reward point calculations.

2.2 Project Objectives

The objectives of this project are to:

- Design a complete relational database for an online store.
- Store and manage information about products, customers, orders, payments, suppliers, and inventory.
- Establish relationships between database tables using primary and foreign keys.
- Reduce data redundancy by applying database normalization.
- Support CRUD (Create, Read, Update, Delete) operations.
- Improve query performance through indexing.
- Implement SQL views for different user roles.
- Automate business rules using database triggers.
- Demonstrate the practical application of SQL DDL and DML statements in MySQL.

2.3 System Scope

The database system supports the core business operations of an online store, including:

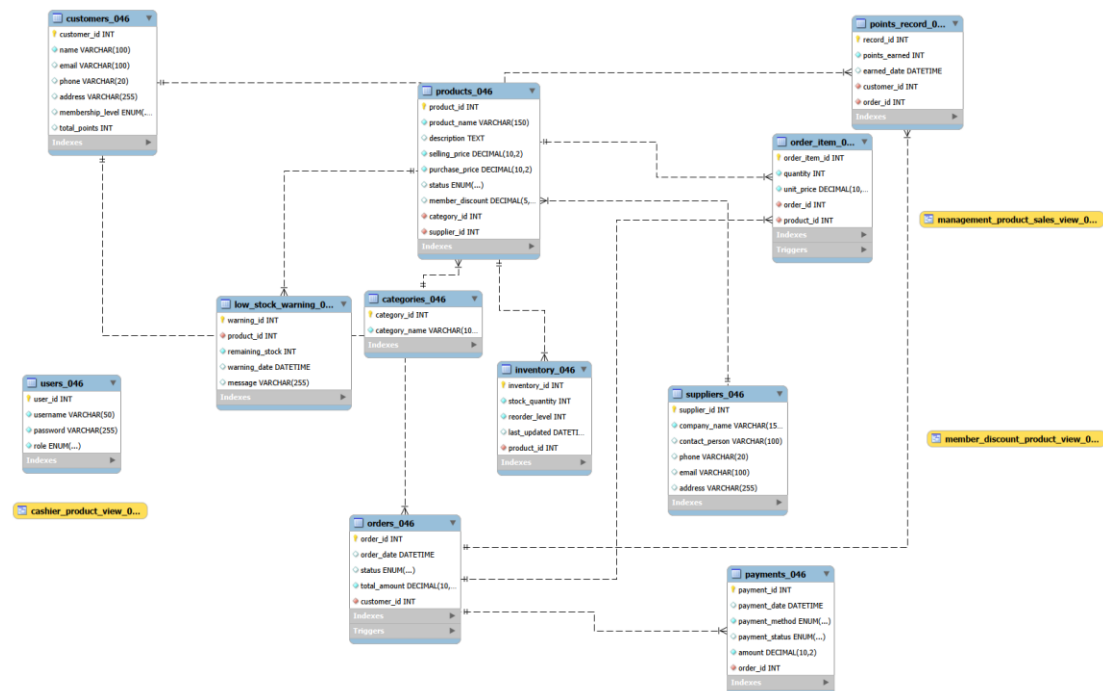
- Product management
- Customer management
- Supplier management
- Inventory management
- Order processing
- Payment management
- Sales reporting
- Member reward point management

The system is intended for multiple types of users, each with different responsibilities and permissions, ensuring that business operations are performed efficiently and securely.

This project serves as a practical implementation of database system concepts,

covering conceptual design, logical design, database implementation, SQL programming, and database management in accordance with the requirements of the Database Systems Composite Experiment.

Part III – Project Design:



Explanation of the E-R Diagram

3.1 Entities and Primary Keys

Entity	Primary Key	Key Attributes
Categories_046	category_id	category_name
Suppliers_046	supplier_id	company_name, contact_person, phone, email, address
Products_046	product_id	product_name, selling_price, purchase_price, status, member_discount
Inventory_046	inventory_id	stock_quantity, reorder_level, last_updated (FK: product_id, unique)
Customers_046	customer_id	name, email, phone, membership_level, total_points
Orders_046	order_id	order_date, status, total_amount (FK: customer_id)
Order_Item_046	order_item_id	quantity, unit_price (FK: order_id, product_id)
Payments_046	payment_id	payment_method, payment_status, amount (FK: order_id, unique)
Users_046	user_id	username, password, role
Points_Record_046	record_id	points_earned, earned_date (FK: customer_id, order_id unique)
Low_Stock_Warning_046	warning_id	remaining_stock, message (FK: product_id)

3.2 Relationships and Cardinalities

Relationship	Cardinality	Participation
Category — Products	1 : N	Category: partial, Product: total (every product must have a category)
Supplier — Products	1 : N	Supplier: partial, Product: total
Product — Inventory	1 : 1	Product: total, Inventory: total (every product has exactly one stock record)
Customer — Orders	1 : N	Customer: partial, Order: total (every order must belong to a customer)
Order — Order_Item	1 : N	Order: total, Order_Item: total
Product — Order_Item	1 : N	Product: partial, Order_Item: total
Order — Payment	1 : 1	Order: partial, Payment: total (a payment must reference exactly one order)
Customer — Points_Record	1 : N	Customer: partial, Points_Record: total
Order — Points_Record	1 : 1	Order: partial, Points_Record: total
Product — Low_Stock_Warning	1 : N	Product: partial, Warning: total

3.3 Normal Form Justification

All tables in this schema satisfy Third Normal Form (3NF), and in practice Boyce-Codd Normal Form (BCNF) as well, for the following reasons:

- 1NF: Every attribute holds a single atomic value (e.g., no comma-separated lists), and each table has a defined primary key (mostly surrogate AUTO_INCREMENT keys).
- 2NF: Since every table uses a single-column surrogate primary key, there are no partial dependencies possible on a composite key — 2NF is satisfied trivially.
- 3NF / BCNF: Non-key attributes depend only on the primary key, not on each other. For example, in Products_046, category_name and company_name are NOT stored redundantly — they live only in Categories_046 and Suppliers_046 and are referenced by foreign key, eliminating transitive dependency. Similarly, Order_Item_046 stores unit_price at the time of sale (a deliberate, justified exception — this is a historical fact of the transaction, not a derivable attribute of Products_046, since prices change over time) rather than repeating full product details.

Part IV– Relational Model

4.1 Relational Schema

The conceptual E-R model of the Online Store Database Management System was converted into a relational database schema and implemented using MySQL. Each entity in the E-R diagram was transformed into a relational table, while the relationships between entities were established using primary keys and foreign keys. The database design follows the Third Normal Form (3NF), ensuring data integrity and minimizing data redundancy.

The relational schema of the database is presented below.

Categories_046

- category_id (PK)
- category_name

Suppliers_046

- supplier_id (PK)
- company_name
- contact_person
- phone
- email
- address

Products_046

- product_id (PK)
- product_name
- description
- selling_price
- purchase_price
- status
- member_discount
- category_id (FK)
- supplier_id (FK)

Inventory_046

- inventory_id (PK)
- stock_quantity
- reorder_level
- last_updated
- product_id (FK)

Customers_046

- customer_id (PK)
- name
- email
- phone
- address
- membership_level
- total_points

Orders_046

- order_id (PK)
- order_date
- status
- total_amount
- customer_id (FK)

Order_Item_046

- order_item_id (PK)
- quantity
- unit_price

- order_id (FK)
- product_id (FK)

Payments_046

- payment_id (PK)
- payment_date
- payment_method
- payment_status
- amount
- order_id (FK)

Users_046

- user_id (PK)
- username
- password
- role

The relationships among these tables are maintained through foreign keys. The Order_Item_046 table serves as an associative table that resolves the many-to-many relationship between Orders_046 and Products_046. The remaining relationships are implemented as one-to-one or one-to-many relationships according to the database requirements.

4.2 SQL DDL (CREATE TABLE Statements)

The database tables were created using SQL Data Definition Language (DDL) statements in MySQL. Each table includes appropriate data types, primary keys, foreign keys, and constraints to ensure data integrity and maintain relationships between entities. Representative CREATE TABLE statements are shown below, followed by screenshots of their successful execution in MySQL Workbench.

The first screenshot shows the SQL editor with the following statements:

```

28
29     category_id INT NOT NULL,
30     supplier_id INT NOT NULL,
31
32     CONSTRAINT fk_product_category
33     FOREIGN KEY (category_id)
34     REFERENCES Categories_046(category_id),
35
36     CONSTRAINT fk_product_supplier
37     FOREIGN KEY (supplier_id)
38     REFERENCES Suppliers_046(supplier_id)
39 }

```

The second screenshot shows the SQL editor with the following statements:

```

52     FOREIGN KEY (product_id)
53     REFERENCES Products_046(product_id)
54 }
55
56 * CREATE TABLE Customers_046 (
57     customer_id INT AUTO_INCREMENT PRIMARY KEY,
58     name VARCHAR(100) NOT NULL,
59     email VARCHAR(100) UNIQUE,
60     phone VARCHAR(20),
61     address VARCHAR(255),
62     membership_level ENUM('Regular', 'Silver', 'Gold', 'VIP')
63     DEFAULT 'Regular',
64     total_purchases INT DEFAULT 0
65 )
66

```

Both screenshots include an 'Output' pane showing the execution results of the SQL statements. The first screenshot shows the execution of the first two statements, and the second screenshot shows the execution of the remaining statements.

#	Time	Action	Message	Duration / Fetch
1	17:30:49	CREATE DATABASE OrderSysDB_046	1 row(s) affected	0.016 sec
2	17:31:12	USE OrderSysDB_046	0 rows(s) affected	0.000 sec
3	17:32:16	CREATE TABLE Categories_046 (category_id INT AUTO_INCREMENT PRIMARY KEY, category_name VARCHAR(100) NOT NULL UNIQUE)	0 rows(s) affected	0.047 sec
4	17:32:48	CREATE TABLE Suppliers_046 (supplier_id INT AUTO_INCREMENT PRIMARY KEY, company_name VARCHAR(150) NOT NULL, contact_person VARCHAR(100) NOT NULL, product_name VARCHAR(150) NOT NULL, description TEXT)	0 rows(s) affected	0.016 sec
5	17:34:35	CREATE TABLE Products_046 (product_id INT AUTO_INCREMENT PRIMARY KEY, product_name VARCHAR(150) NOT NULL, description TEXT)	0 rows(s) affected	0.016 sec
6	17:35:07	CREATE TABLE Products_046 (product_id INT AUTO_INCREMENT PRIMARY KEY, product_name VARCHAR(150) NOT NULL, description TEXT)	0 rows(s) affected	0.047 sec

#	Time	Action	Message	Duration / Fetch
1	17:30:49	CREATE DATABASE OrderSysDB_046	1 row(s) affected	0.016 sec
2	17:31:12	USE OrderSysDB_046	0 rows(s) affected	0.000 sec
3	17:32:16	CREATE TABLE Categories_046 (category_id INT AUTO_INCREMENT PRIMARY KEY, category_name VARCHAR(100) NOT NULL UNIQUE)	0 rows(s) affected	0.047 sec
4	17:32:48	CREATE TABLE Suppliers_046 (supplier_id INT AUTO_INCREMENT PRIMARY KEY, company_name VARCHAR(150) NOT NULL, contact_person VARCHAR(100) NOT NULL, product_name VARCHAR(150) NOT NULL, description TEXT)	0 rows(s) affected	0.016 sec
5	17:34:35	CREATE TABLE Products_046 (product_id INT AUTO_INCREMENT PRIMARY KEY, product_name VARCHAR(150) NOT NULL, description TEXT)	0 rows(s) affected	0.016 sec
6	17:35:07	CREATE TABLE Products_046 (product_id INT AUTO_INCREMENT PRIMARY KEY, product_name VARCHAR(150) NOT NULL, description TEXT)	0 rows(s) affected	0.047 sec
7	17:35:50	CREATE TABLE Inventory_046 (inventory_id INT AUTO_INCREMENT PRIMARY KEY, stock_quantity INT NOT NULL DEFAULT 0, reorder_level INT, last_updated DATETIME)	0 rows(s) affected	0.047 sec
8	17:36:28	CREATE TABLE Customers_046 (customer_id INT AUTO_INCREMENT PRIMARY KEY, name VARCHAR(100) NOT NULL, email VARCHAR(100) NOT NULL, phone VARCHAR(20), address VARCHAR(255), membership_level ENUM('Regular', 'Silver', 'Gold', 'VIP') DEFAULT 'Regular', total_purchases INT DEFAULT 0)	0 rows(s) affected	0.032 sec

SCHEMAS

Filter objects

- db_uni046
 - Tables
 - Orders
 - Products
 - Suppliers
 - Customers
 - Payments
 - Inventory
 - Categories
 - Functions
 - Views
 - Stored Procedures
 - Triggers

Administration Schemas Information

Schemas: db_uni046

```

41
42 CREATE TABLE Inventory_046 (
43     Inventory_id INT AUTO_INCREMENT PRIMARY KEY,
44     stock_quantity INT NOT NULL DEFAULT 0,
45     product_id INT NOT NULL DEFAULT 0,
46     last_updated DATETIME DEFAULT CURRENT_TIMESTAMP
47     ON UPDATE CURRENT_TIMESTAMP,
48
49     product_id INT NOT NULL UNIQUE,
50
51     CONSTRAINT fk_inventory_product
52     FOREIGN KEY (product_id)
53     REFERENCES Products_046(product_id)
54 );

```

Output

#	Time	Action	Message	Duration / Fetch
1	17:30:43	CREATE DATABASE OnlineStoreDB_046	1 row(s) affected	0.016 sec
2	17:31:12	USE OnlineStoreDB_046	0 row(s) affected	0.000 sec
3	17:32:16	CREATE TABLE Categories_046 (category_id INT AUTO_INCREMENT PRIMARY KEY, category_name VARCHAR(100) NOT NULL UNIQUE)	0 row(s) affected	0.047 sec
4	17:32:48	CREATE TABLE Suppliers_046 (supplier_id INT AUTO_INCREMENT PRIMARY KEY, company_name VARCHAR(100) NOT NULL, contact_person VARCHAR(100) NOT NULL, description TEXT)	0 row(s) affected	0.016 sec
5	17:34:30	CREATE TABLE Products_046 (product_id INT AUTO_INCREMENT PRIMARY KEY, product_name VARCHAR(100) NOT NULL, description TEXT)	Error Code: 1024: Failed to open the referenced table 'categories_pay'	0.016 sec
6	17:34:37	CREATE TABLE Products_046 (product_id INT AUTO_INCREMENT PRIMARY KEY, product_name VARCHAR(100) NOT NULL, description TEXT)	0 row(s) affected	0.047 sec
7	17:35:50	CREATE TABLE Inventory_046 (inventory_id INT AUTO_INCREMENT PRIMARY KEY, stock_quantity INT NOT NULL DEFAULT 0, product_id INT NOT NULL, last_updated DATETIME DEFAULT CURRENT_TIMESTAMP)	0 row(s) affected	0.047 sec

SCHEMAS

Filter objects

- db_uni046
 - Tables
 - Orders
 - Products
 - Suppliers
 - Customers
 - Payments
 - Inventory
 - Categories
 - Functions
 - Views
 - Stored Procedures
 - Triggers

Administration Schemas Information

Schemas: db_uni046

```

98
99 CREATE TABLE Payments_XXX (
100     payment_id INT AUTO_INCREMENT PRIMARY KEY,
101     payment_date DATETIME DEFAULT CURRENT_TIMESTAMP,
102
103     payment_method ENUM(
104         'Cash',
105         'Credit Card',
106         'Debit Card',
107         'Online Payment'
108     ) NOT NULL,
109
110     payment_status ENUM(
111         'Pending',
112         'Paid',
113         'Failed',
114         'Refunded'
115     ) DEFAULT 'Pending',
116
117     amount DECIMAL(10,2) NOT NULL,
118
119     order_id INT NOT NULL UNIQUE,
120
121     CONSTRAINT fk_payment_order
122     FOREIGN KEY (order_id)
123     REFERENCES Orders_XXX(order_id)
124 );

```

Output

#	Time	Action	Message	Duration / Fetch
1	17:30:43	CREATE DATABASE OnlineStoreDB_046	1 row(s) affected	0.016 sec
2	17:31:12	USE OnlineStoreDB_046	0 row(s) affected	0.000 sec
3	17:32:16	CREATE TABLE Categories_046 (category_id INT AUTO_INCREMENT PRIMARY KEY, category_name VARCHAR(100) NOT NULL UNIQUE)	0 row(s) affected	0.047 sec
4	17:32:48	CREATE TABLE Suppliers_046 (supplier_id INT AUTO_INCREMENT PRIMARY KEY, company_name VARCHAR(100) NOT NULL, contact_person VARCHAR(100) NOT NULL, description TEXT)	0 row(s) affected	0.016 sec
5	17:34:30	CREATE TABLE Products_046 (product_id INT AUTO_INCREMENT PRIMARY KEY, product_name VARCHAR(100) NOT NULL, description TEXT)	Error Code: 1024: Failed to open the referenced table 'categories_pay'	0.016 sec
6	17:34:37	CREATE TABLE Products_046 (product_id INT AUTO_INCREMENT PRIMARY KEY, product_name VARCHAR(100) NOT NULL, description TEXT)	0 row(s) affected	0.047 sec
7	17:35:50	CREATE TABLE Inventory_046 (inventory_id INT AUTO_INCREMENT PRIMARY KEY, stock_quantity INT NOT NULL DEFAULT 0, product_id INT NOT NULL, last_updated DATETIME DEFAULT CURRENT_TIMESTAMP)	0 row(s) affected	0.047 sec
8	17:36:28	CREATE TABLE Customers_046 (customer_id INT AUTO_INCREMENT PRIMARY KEY, name VARCHAR(100) NOT NULL, email VARCHAR(100) NOT NULL, phone VARCHAR(20) NOT NULL)	0 row(s) affected	0.032 sec
9	17:37:08	CREATE TABLE Orders_046 (order_id INT AUTO_INCREMENT PRIMARY KEY, order_date DATETIME DEFAULT CURRENT_TIMESTAMP, customer_id INT NOT NULL, product_id INT NOT NULL, quantity INT NOT NULL, total_price DECIMAL(10,2) NOT NULL)	0 row(s) affected	0.015 sec

```

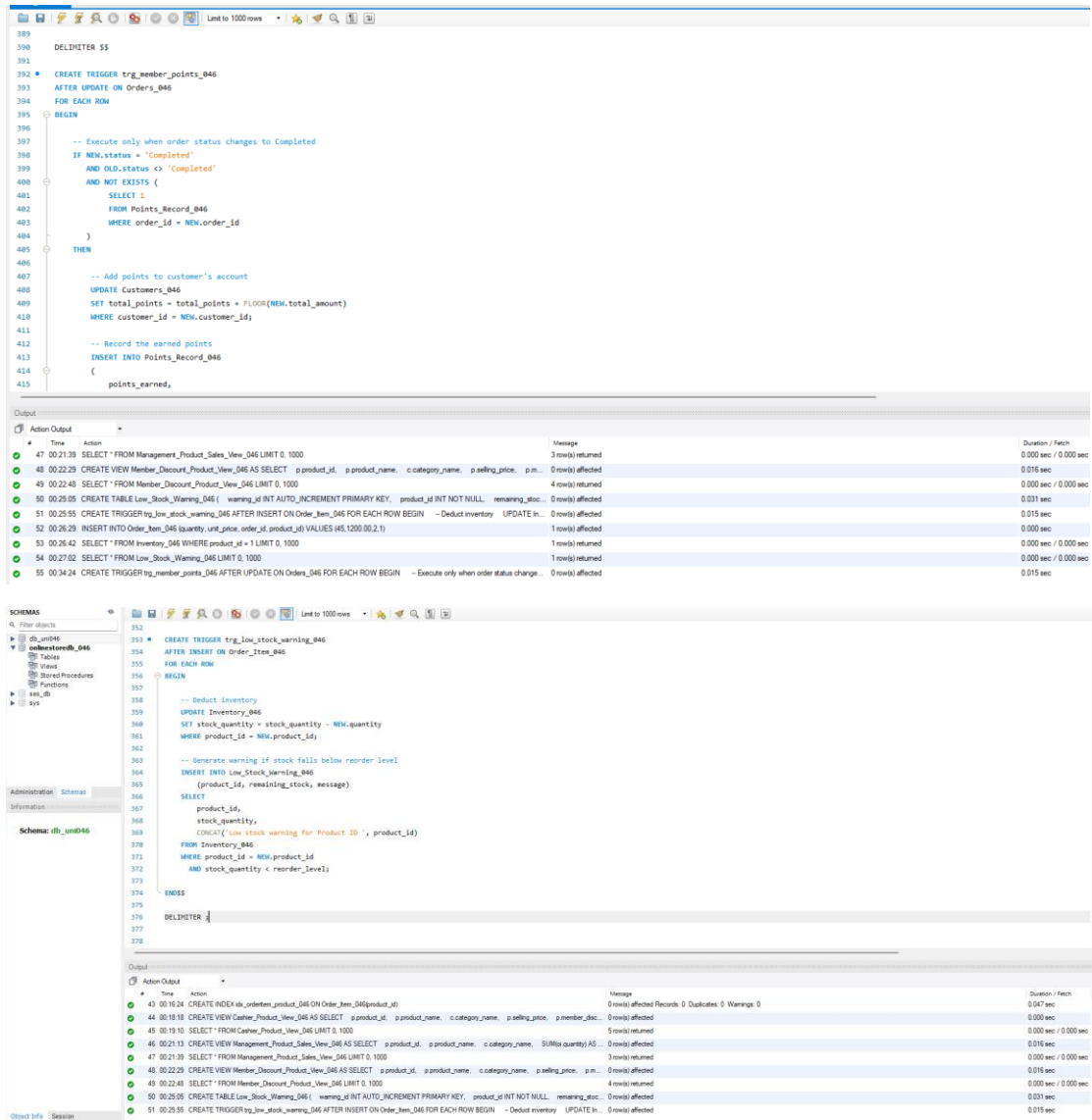
276
277 CREATE VIEW Cashier_Product_View_046 AS
278 SELECT
279     p.product_id,
280     p.product_name,
281     c.category_name,
282     p.selling_price,
283     p.member_discount,
284     i.stock_quantity,
285     p.status
286 FROM Products_046 p
287 JOIN Categories_046 c
288     ON p.category_id = c.category_id
289 JOIN Inventory_046 i
290     ON p.product_id = i.product_id
291 WHERE p.status = 'Available';

```

Output

#	Time	Action	Message	Duration / Fetch
36	18:43:43	SELECT * FROM Payments_046 LIMIT 0, 1000	5 row(s) returned	0.000 sec / 0.000 sec
37	18:43:45	SELECT * FROM Payments_046 LIMIT 0, 1000	2 row(s) returned	0.000 sec / 0.000 sec
38	18:46:34	CREATE INDEX idx_product_name_046 ON Products_046(product_name)	0 row(s) affected Records: 0 Duplicates: 0 Warnings: 0	0.046 sec
39	18:46:41	SELECT * FROM Products_046 WHERE product_name = 'Laptop' LIMIT 0, 1000	1 row(s) returned	0.000 sec / 0.000 sec
40	00:15:49	CREATE INDEX idx_customer_email_046 ON Customers_046(email)	0 row(s) affected Records: 0 Duplicates: 0 Warnings: 0	0.047 sec
41	00:15:50	CREATE INDEX idx_order_date_046 ON Orders_046(order_date)	0 row(s) affected Records: 0 Duplicates: 0 Warnings: 0	0.079 sec
42	00:16:07	CREATE INDEX idx_payment_status_046 ON Payments_046(payment_status)	0 row(s) affected Records: 0 Duplicates: 0 Warnings: 0	0.046 sec
43	00:16:24	CREATE INDEX idx_orderitem_product_046 ON Orderitem_046(product_id)	0 row(s) affected Records: 0 Duplicates: 0 Warnings: 0	0.047 sec
44	00:16:18	CREATE VIEW Cashier_Product_View_046 AS SELECT p.product_id, p.product_name, c.category_name, p.selling_price, p.member_discount, i.stock_quantity, p.status	0 row(s) affected	0.000 sec

4.3 SQL INDEX



The following figures present screenshots of the SQL statements executed in MySQL Workbench during the implementation of the Online Store Database Management System. These screenshots demonstrate the successful creation of the database objects, including tables, views, and triggers. Each SQL statement was executed without errors, ensuring that the database schema, constraints, and automation features were implemented correctly.

4.6 10 Single-Table Queries

```
547 • SELECT * FROM Customers_046;
548
549
550
```

Result Grid							
Filter Rows:							
Edit: Export/Import: Wrap Cell Content: I							
customer_id	name	email	phone	address	membership_level	total_points	
1	Alice Johnson	alice@gmail.com	1111111111	California	Gold	250	
2	Bob Williams	bob@gmail.com	2222222222	Texas	Gold	75	
3	Charlie Brown	charlie@gmail.com	3333333333	Florida	Silver	120	
4	Diana Miller	diana@gmail.com	4444444444	Nevada	VIP	1400	
5	Ethan Davis	ethan@gmail.com	5555555555	Arizona	Regular	0	
*	NULL	NULL	NULL	NULL	NULL	NULL	

Customers_046 1 x

Output				
Action Output				
#	Time	Action	Message	Duration / Fetch
✓ 1	21:58:57	USE OnlineStoreDB_046	0 row(s) affected	0.000 sec
✓ 2	21:59:20	SELECT * FROM Customers_046 LIMIT 0, 1000	5 row(s) returned	0.015 sec / 0.000 sec

```
550 • SELECT *
551 FROM Products_046
552 WHERE status = 'Available';
```

Result Grid		Filter Rows:		Edit:		Export/Import:		Wrap Cell Content:		
	product_id	product_name	description	selling_price	purchase_price	status	member_discount	category_id	supplier_id	
▶	1	Laptop	15-inch gaming laptop	1300.00	950.00	Available	10.00	1	1	
	2	Smartphone	Android smartphone	800.00	650.00	Available	5.00	1	1	
	3	T-Shirt	Cotton T-Shirt	25.00	15.00	Available	0.00	2	2	
	4	Database Book	Learning SQL	50.00	30.00	Available	15.00	3	3	
	5	Football	Professional football	40.00	22.00	Available	8.00	5	5	
	6	Wireless Mouse	Bluetooth wireless mouse	35.00	20.00	Available	5.00	1	1	
*	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	

Products_046 2 x

Output				
Action Output				
#	Time	Action	Message	Duration / Fetch
✓ 1	21:58:57	USE OnlineStoreDB_046	0 row(s) affected	0.000 sec
✓ 2	21:59:20	SELECT * FROM Customers_046 LIMIT 0, 1000	5 row(s) returned	0.015 sec / 0.000 sec
✓ 3	22:02:10	SELECT * FROM Products_046 WHERE status = 'Available' LI...	6 row(s) returned	0.016 sec / 0.000 sec

```
555 • SELECT * FROM Suppliers_046;
```

Result Grid					
Filter Rows:					
Edit: Export/Import: Wrap Cell Content: I					
supplier_id	company_name	contact_person	phone	email	address
1	Tech World Ltd.	John Smith	1234567890	john@techworld.com	New York
2	Fashion Hub	Emma Brown	2233445566	emma@fashionhub.com	Los Angeles
3	Book House	David Wilson	3344556677	david@bookhouse.com	Chicago
4	Home Supplies Inc.	Sophia Davis	4455667788	sophia@homesupplies.com	Houston
5	Sport Zone	Michael Lee	5566778899	michael@sportzone.com	Miami
*	NULL	NULL	NULL	NULL	NULL

Suppliers_046 3 x

Output				
Action Output				
#	Time	Action	Message	Duration / Fetch
✓ 2	21:59:20	SELECT * FROM Customers_046 LIMIT 0, 1000	5 row(s) returned	0.015 sec / 0.000 sec
✓ 3	22:02:10	SELECT * FROM Products_046 WHERE status = 'Available' ...	6 row(s) returned	0.016 sec / 0.000 sec
✓ 4	22:02:33	SELECT * FROM Suppliers_046 LIMIT 0, 1000	5 row(s) returned	0.031 sec / 0.000 sec

```

557 • SELECT *
558 FROM Products_046
559 WHERE member_discount > 0;

```

product_id	product_name	description	selling_price	purchase_price	status	member_discount	category_id	supplier_id
1	Laptop	15-inch gaming laptop	1300.00	950.00	Available	10.00	1	1
2	Smartphone	Android smartphone	800.00	650.00	Available	5.00	1	1
4	Database Book	Learning SQL	50.00	30.00	Available	15.00	3	3
5	Football	Professional football	40.00	22.00	Available	8.00	5	5
6	Wireless Mouse	Bluetooth wireless mouse	35.00	20.00	Available	5.00	1	1
NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL

Products_046 4 x

Apply

Revert

Output

Action Output

#	Time	Action	Message	Duration / Fetch
3	22:02:10	SELECT * FROM Products_046 WHERE status = 'Available' ...	6 row(s) returned	0.016 sec / 0.000 sec
4	22:02:33	SELECT * FROM Suppliers_046 LIMIT 0, 1000	5 row(s) returned	0.031 sec / 0.000 sec
5	22:02:46	SELECT * FROM Products_046 WHERE member_discount > ...	5 row(s) returned	0.000 sec / 0.000 sec

```

566 • SELECT *
567 FROM Orders_046
568 WHERE status = 'Completed';

```

order_id	order_date	status	total_amount	customer_id
1	2026-06-20 00:00:00	Completed	1250.00	1
2	2026-06-21 00:00:00	Completed	25.00	2
3	2026-06-22 00:00:00	Completed	90.00	3
4	2026-06-23 00:00:00	Completed	800.00	4
NULL	NULL	NULL	NULL	NULL

```

570 • SELECT *
571 FROM Payments_046
572 WHERE payment_status = 'Paid';

```

payment_id	payment_date	payment_method	payment_status	amount	order_id
1	2026-06-20 00:00:00	Credit Card	Paid	1250.00	1
3	2026-06-22 00:00:00	Debit Card	Paid	90.00	3
NULL	NULL	NULL	NULL	NULL	NULL

Payments_046 6 x

Apply

Revert

Output

Action Output

#	Time	Action	Message	Duration / Fetch
5	22:02:46	SELECT * FROM Products_046 WHERE member_discount > ...	5 row(s) returned	0.000 sec / 0.000 sec
6	22:04:43	SELECT * FROM Orders_046 WHERE status = 'Completed' LI...	4 row(s) returned	0.015 sec / 0.000 sec
7	22:04:55	SELECT * FROM Payments_046 WHERE payment_status = '...	2 row(s) returned	0.016 sec / 0.000 sec

575 • **SELECT** *
576 **FROM** Products_046
577 **ORDER BY** selling_price DESC;

	product_id	product_name	description	selling_price	purchase_price	status	member_discount	category_id	supplier_id
▶	1	Laptop	15-inch gaming laptop	1300.00	950.00	Available	10.00	1	1
	2	Smartphone	Android smartphone	800.00	650.00	Available	5.00	1	1
	4	Database Book	Learning SQL	50.00	30.00	Available	15.00	3	3
	5	Football	Professional football	40.00	22.00	Available	8.00	5	5
	6	Wireless Mouse	Bluetooth wireless mouse	35.00	20.00	Available	5.00	1	1
	3	T-Shirt	Cotton T-Shirt	25.00	15.00	Available	0.00	2	2
*	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL

Products_046 7 ×

Apply Revert

Output

Action Output

#	Time	Action	Message	Duration / Fetch
✓	6 22:04:43	SELECT * FROM Orders_046 WHERE status = 'Completed' LI...	4 row(s) returned	0.015 sec / 0.000 sec
✓	7 22:04:55	SELECT * FROM Payments_046 WHERE payment_status = '...	2 row(s) returned	0.016 sec / 0.000 sec
✓	8 22:05:10	SELECT * FROM Products_046 ORDER BY selling_price DE...	6 row(s) returned	0.000 sec / 0.000 sec

580 • **SELECT** *
581 **FROM** Users_046
582 **ORDER BY** role;

	user_id	username	password	role
▶	1	manager1	manager123	Manager
	2	cashier1	cashier123	Cashier
	3	inventory1	inventory123	Inventory Clerk
*	NULL	NULL	NULL	NULL

Users_046 8 ×

Apply Revert

Output

Action Output

#	Time	Action	Message	Duration / Fetch
✓	7 22:04:55	SELECT * FROM Payments_046 WHERE payment_status = '...	2 row(s) returned	0.016 sec / 0.000 sec
✓	8 22:05:10	SELECT * FROM Products_046 ORDER BY selling_price DE...	6 row(s) returned	0.000 sec / 0.000 sec
✓	9 22:05:24	SELECT * FROM Users_046 ORDER BY role LIMIT 0, 1000	3 row(s) returned	0.016 sec / 0.000 sec

4.7 5 Complex Multi-Table Queries

```

586 • SELECT
587     c.customer_id,
588     c.name,
589     o.order_id,
590     o.order_date,
591     o.total_amount,
592     o.status
593 FROM Customers_046 c
594 JOIN Orders_046 o
595 ON c.customer_id = o.customer_id;

```

Result Grid | Filter Rows: | Export: | Wrap Cell Content: |

	customer_id	name	order_id	order_date	total_amount	status
▶	1	Alice Johnson	1	2026-06-20 00:00:00	1250.00	Completed
	2	Bob Williams	2	2026-06-21 00:00:00	25.00	Completed
	3	Charlie Brown	3	2026-06-22 00:00:00	90.00	Completed
	4	Diana Miller	4	2026-06-23 00:00:00	800.00	Completed
	5	Ethan Davis	5	2026-06-24 00:00:00	40.00	Cancelled

Result Grid | Form Editor

Result 9 × Read Only

Output

Action Output

#	Time	Action	Message	Duration / Fetch
✓ 8	22:05:10	SELECT * FROM Products_046 ORDER BY selling_price DE...	6 row(s) returned	0.000 sec / 0.000 sec
✓ 9	22:05:24	SELECT * FROM Users_046 ORDER BY role LIMIT 0, 1000	3 row(s) returned	0.016 sec / 0.000 sec
✓ 10	22:08:14	SELECT c.customer_id, c.name, o.order_id, o.order...	5 row(s) returned	0.000 sec / 0.000 sec

```

598 • SELECT
599     p.product_name,
600     c.category_name,
601     s.company_name,
602     p.selling_price,
603     p.status
604 FROM Products_046 p
605 JOIN Categories_046 c
606 ON p.category_id = c.category_id
607 JOIN Suppliers_046 s
608 ON p.supplier_id = s.supplier_id;

```

Result Grid | Filter Rows: | Export: | Wrap Cell Content: |

	product_name	category_name	company_name	selling_price	status
▶	Database Book	Books	Book House	50.00	Available
	T-Shirt	Clothing	Fashion Hub	25.00	Available
	Laptop	Electronics	Tech World Ltd.	1300.00	Available
	Smartphone	Electronics	Tech World Ltd.	800.00	Available
	Wireless Mouse	Electronics	Tech World Ltd.	35.00	Available
	Football	Sports	Sport Zone	40.00	Available

Result Grid | Form Editor

Result 10 × Read Only

Output

Action Output

#	Time	Action	Message	Duration / Fetch
✓ 9	22:05:24	SELECT * FROM Users_046 ORDER BY role LIMIT 0, 1000	3 row(s) returned	0.016 sec / 0.000 sec
✓ 10	22:08:14	SELECT c.customer_id, c.name, o.order_id, o.order...	5 row(s) returned	0.000 sec / 0.000 sec
✓ 11	22:08:32	SELECT p.product_name, c.category_name, s.comp...	6 row(s) returned	0.000 sec / 0.000 sec

```

610 • SELECT
611     o.order_id,
612     c.name,
613     p.product_name,
614     oi.quantity,
615     oi.unit_price
616 FROM Orders_046 o
617 JOIN Customers_046 c
618 ON o.customer_id = c.customer_id
619 JOIN Order_Item_046 oi
620 ON o.order_id = oi.order_id
621 JOIN Products_046 p
622 ON oi.product_id = p.product_id;

```

Result Grid					
Filter Rows:					
Export: Wrap Cell Content:					
order_id	name	product_name	quantity	unit_price	
1	Alice Johnson	Laptop	1	1200.00	
1	Alice Johnson	Database Book	1	50.00	
2	Bob Williams	T-Shirt	1	25.00	
3	Charlie Brown	Database Book	1	50.00	
3	Charlie Brown	Football	1	40.00	
4	Diana Miller	Smartphone	1	800.00	
5	Ethan Davis	Football	1	40.00	
2	Bob Williams	Laptop	45	1200.00	

Result 11 ×

Read Only

Output

Action Output

#	Time	Action	Message	Duration / Fetch
✓ 10	22:08:14	SELECT c.customer_id, c.name, o.order_id, o.order...	5 row(s) returned	0.000 sec / 0.000 sec
✓ 11	22:08:32	SELECT p.product_name, c.category_name, s.compa...	6 row(s) returned	0.000 sec / 0.000 sec
✓ 12	22:08:49	SELECT o.order_id, c.name, p.product_name, oi.qu...	8 row(s) returned	0.016 sec / 0.000 sec

```

624 • SELECT
625     c.customer_id,
626     c.name,
627     SUM(o.total_amount) AS Total_Spent
628 FROM Customers_046 c
629 JOIN Orders_046 o
630 ON c.customer_id = o.customer_id
631 GROUP BY c.customer_id, c.name
632 ORDER BY Total_Spent DESC;

```

Result Grid			
Filter Rows:			
Export: Wrap Cell Content:			
customer_id	name	Total_Spent	
1	Alice Johnson	1250.00	
4	Diana Miller	800.00	
3	Charlie Brown	90.00	
5	Ethan Davis	40.00	
2	Bob Williams	25.00	

Result 12 ×

Read Only

Output

Action Output

#	Time	Action	Message	Duration / Fetch
✓ 11	22:08:32	SELECT p.product_name, c.category_name, s.compa...	6 row(s) returned	0.000 sec / 0.000 sec
✓ 12	22:08:49	SELECT o.order_id, c.name, p.product_name, oi.qu...	8 row(s) returned	0.016 sec / 0.000 sec
✓ 13	22:09:16	SELECT c.customer_id, c.name, SUM(o.total_amount)...	5 row(s) returned	0.000 sec / 0.000 sec

```

634 • SELECT
635     product_id,
636     quantity
637 FROM Order_Item_046
638 WHERE quantity >
639 (
640     SELECT AVG(quantity)
641     FROM Order_Item_046
642 );

```

Result Grid		Filter Rows:	Export:	Wrap Cell Content:
product_id	quantity			
1	45			

Order_Item_046 13 x

Output

Action Output

#	Time	Action	Message	Duration / Fetch
✓ 12	22:08:49	SELECT o.order_id, c.name, p.product_name, oi.qu...	8 row(s) returned	0.016 sec / 0.000 sec
✓ 13	22:09:16	SELECT c.customer_id, c.name, SUM(o.total_amount)...	5 row(s) returned	0.000 sec / 0.000 sec
✓ 14	22:09:30	SELECT product_id, quantity FROM Order_Item_046 W...	1 row(s) returned	0.000 sec / 0.000 sec

4.8 CRUD operations

```

643
644 • INSERT INTO Customers_046
645     (name, email, phone, address, membership_level)
646     VALUES
647     ('John Smith',
648     'john@gmail.com',
649     '1234567890',
650     'New York',
651     'Gold');
652

```

Output

Action Output

#	Time	Action	Message	Duration / Fetch
✓ 13	22:09:16	SELECT c.customer_id, c.name, SUM(o.total_amount)...	5 row(s) returned	0.000 sec / 0.000 sec
✓ 14	22:09:30	SELECT product_id, quantity FROM Order_Item_046 W...	1 row(s) returned	0.000 sec / 0.000 sec
✓ 15	22:30:48	INSERT INTO Customers_046 (name, email, phone, address, ...	1 row(s) affected	0.000 sec

652
653 • **SELECT ***
654 **FROM Customers_046;**
655

customer_id	name	email	phone	address	membership_level	total_points
1	Alice Johnson	alice@gmail.com	1111111111	California	Gold	250
2	Bob Williams	bob@gmail.com	2222222222	Texas	Gold	75
3	Charlie Brown	charlie@gmail.com	3333333333	Florida	Silver	120
4	Diana Miller	diana@gmail.com	4444444444	Nevada	VIP	1400
5	Ethan Davis	ethan@gmail.com	5555555555	Arizona	Regular	0
7	John Smith	john@gmail.com	1234567890	New York	Gold	0

Customers_046 14 x

Output

#	Time	Action	Message	Duration / Fetch
15	22:30:48	INSERT INTO Customers_046 (name, email, phone, address, ...)	1 row(s) affected	0.000 sec
16	22:32:09	SELECT * FROM Customers_046 LIMIT 0, 1000	6 row(s) returned	0.000 sec / 0.000 sec

656
657 • **UPDATE Customers_046**
658 **SET** phone = '9876543210',
659 **membership_level** = 'VIP'
660 **WHERE** customer_id = 1;

Output

#	Time	Action	Message	Duration / Fetch
16	22:32:09	SELECT * FROM Customers_046 LIMIT 0, 1000	6 row(s) returned	0.000 sec / 0.000 sec
17	22:32:25	UPDATE Customers_046 SET phone = '9876543210', mem...	1 row(s) affected Rows matched: 1 Changed: 1 Warnings: 0	0.015 sec

536 • **DELETE FROM Customers_046**
537 **WHERE** customer_id = 6;

Output

#	Time	Action	Message	Duration / Fetch
10	22:08:14	SELECT c.customer_id, c.name, o.order_id, o.order...	5 row(s) returned	0.000 sec / 0.000 sec
11	22:08:32	SELECT p.product_name, c.category_name, s.comp...	6 row(s) returned	0.000 sec / 0.000 sec
12	22:08:49	SELECT o.order_id, c.name, p.product_name, oi.qu...	8 row(s) returned	0.016 sec / 0.000 sec
13	22:09:16	SELECT c.customer_id, c.name, SUM(o.total_amount)...	5 row(s) returned	0.000 sec / 0.000 sec
14	22:09:30	SELECT product_id, quantity FROM Order_Item_046 W...	1 row(s) returned	0.000 sec / 0.000 sec
15	22:30:48	INSERT INTO Customers_046 (name, email, phone, address, ...)	1 row(s) affected	0.000 sec
16	22:32:09	SELECT * FROM Customers_046 LIMIT 0, 1000	6 row(s) returned	0.000 sec / 0.000 sec
17	22:32:25	UPDATE Customers_046 SET phone = '9876543210', mem...	1 row(s) affected Rows matched: 1 Changed: 1 Warnings: 0	0.015 sec
18	22:32:37	DELETE FROM Customers_046 WHERE customer_id = 1	Error Code: 1451. Cannot delete or update a parent row: a for...	0.015 sec
19	22:32:51	DELETE FROM Customers_046 WHERE customer_id = 2	Error Code: 1451. Cannot delete or update a parent row: a for...	0.031 sec
20	22:33:37	DELETE FROM Customers_046 WHERE customer_id = 6	0 row(s) affected	0.000 sec

Part V – Project Conclusion

The Online Store Database Management System project provided valuable practical experience in designing, implementing, and managing a relational database using MySQL. Throughout this project, the knowledge learned in the Database Systems course was applied to develop a complete database solution for an online store. The main objective of the project was to design a reliable database that can efficiently manage products, customers, suppliers, inventory, orders, payments, and users while maintaining data accuracy and consistency. By completing this project, I

gained a better understanding of how database systems are used to support real business operations.

The project started with analyzing the system requirements and identifying the entities, attributes, and relationships needed for the online store. An Entity-Relationship (E-R) diagram was designed to represent the conceptual structure of the database. After completing the conceptual design, it was converted into a relational model suitable for implementation in MySQL. The database tables were normalized to Third Normal Form (3NF) to reduce data redundancy and improve data integrity. Primary keys, foreign keys, and other constraints were carefully defined to establish relationships between tables and ensure that invalid or inconsistent data could not be stored.

After the database design was completed, the implementation phase was carried out using SQL Data Definition Language (DDL) statements. All required tables, indexes, views, and triggers were successfully created. Sample data was inserted into the tables to test the database and verify that all relationships and constraints functioned correctly. CRUD operations were also developed to demonstrate how records can be added, retrieved, updated, and deleted. These operations represent the basic functions required for managing data in any database system.

To meet the project requirements, different SQL queries were created and tested. These included single-table queries and complex multi-table queries using JOIN, GROUP BY, and subqueries. The queries provide useful information for different users of the system, such as managers, cashiers, and inventory clerks. In addition, database views were created to simplify access to information for different user roles while hiding unnecessary or sensitive data. Triggers were implemented to automate important tasks, such as updating inventory levels after sales and calculating customer membership points when completed orders are processed.

As an additional feature, a simple graphical user interface was developed using Python Flask to connect with the MySQL database. The application allows users to perform common database operations through an easy-to-use interface instead of manually entering SQL commands. This improved the usability of the system and demonstrated how a database can be integrated into a software application.

Overall, this project successfully achieved its objectives and fulfilled the requirements of the Database Systems composite experiment. It improved my understanding of database analysis, conceptual design, relational modeling, SQL programming, normalization, and database implementation. More importantly, it gave me practical experience in developing a real-world database application from the initial planning stage to the final implementation and testing. The knowledge and skills gained during this project will be valuable for future academic work and software development projects that require efficient and well-designed database systems.

System Demonstration:

	product_id	product_name	stock_quantity	reorder_level	last_updated
1	Laptop	5	10	2026-07-02 00:26:29	
2	Smartphone	80	15	2026-07-01 18:37:35	
3	T-Shirt	120	20	2026-07-01 18:37:35	
4	Database Book	40	10	2026-07-01 18:37:35	
5	Football	70	15	2026-07-01 18:37:35	
6	Wireless Mouse	100	20	2026-07-02 01:08:07	

Show Low Stock Only

Online Store Manager - manager1 (Manager)

Products Inventory Customers Orders

Search:

	product_id	product_name	stock_quantity	reorder_level	last_updated
1	Laptop	5	10	2026-07-02 00:26:29	
2	Smartphone	80	15	2026-07-01 18:37:35	
3	T-Shirt	120	20	2026-07-01 18:37:35	
4	Database Book	40	10	2026-07-01 18:37:35	
5	Football	70	15	2026-07-01 18:37:35	
6	Wireless Mouse	100	20	2026-07-02 01:08:07	

Show Low Stock Only

Online Store Manager - manager1 (Manager)

Products Inventory Customers Orders

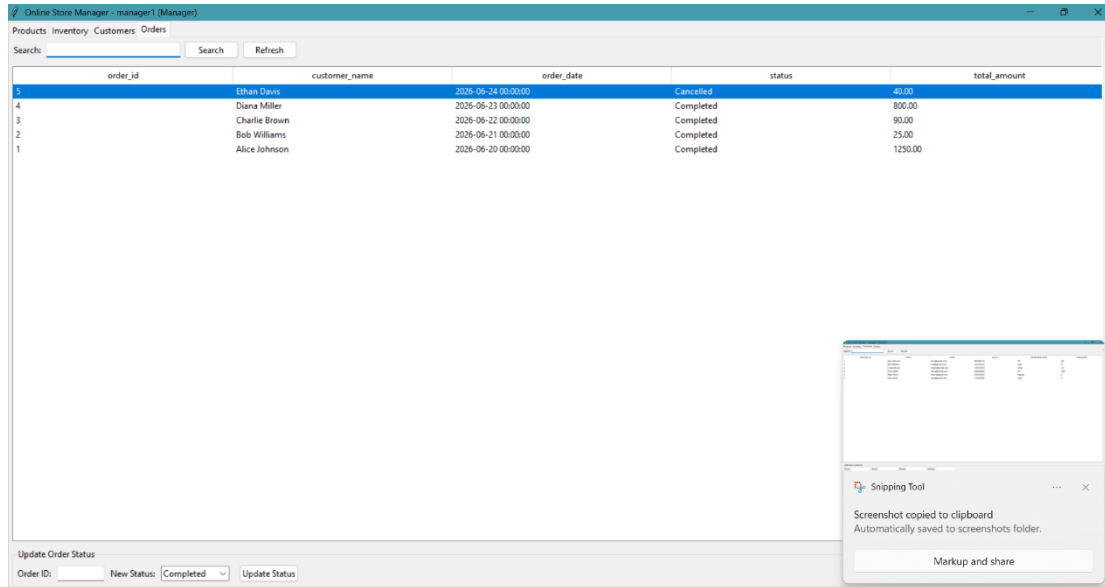
Search:

	customer_id	name	email	phone	membership_level	total_points
1		Alice Johnson	alice@gmail.com	9676543210	VIP	230
2		Bob Williams	bob@gmail.com	2222222222	Gold	75
3		Charlie Brown	charlie@gmail.com	3333333333	Silver	120
4		Diana Miller	diana@gmail.com	4444444444	VIP	1400
5		Ethan Davis	ethan@gmail.com	5555555555	Regular	0
7		John Smith	john@gmail.com	1234567890	Gold	0

Add New Customer

Name Email Phone Address

Add Customer



The Online Store Management System was implemented as a Python/Tkinter desktop application connected to the OnlineStoreDB_046 MySQL database. The application demonstrates all required CRUD operations, views, and triggers through five functional modules (tabs), accessible after authenticating against the Users_046 table.

1. Login

On launch, the user is prompted for a username and password, which are validated against the Users_046 table (role: Manager, Cashier, or Inventory Clerk). This demonstrates a basic SELECT query with a WHERE clause on two columns and simulates role-based system access.

2. Products Tab

Displays the full product catalog joined with category name and live stock quantity (a 3-table JOIN across Products_046, Categories_046, and Inventory_046). Staff can search products by name and add new products through a form, which inserts into Products_046 and simultaneously creates the corresponding Inventory_046 record — demonstrating Create and Read operations across two related tables in a single transaction.

3. Inventory Tab

Lists current stock levels and reorder thresholds for every product. A "Show Low Stock Only" button filters to items where stock_quantity is below reorder_level, giving the Inventory Clerk an at-a-glance view of what needs restocking. This tab is directly linked to the low-stock warning trigger: whenever an order reduces a product's stock below its threshold, the resulting warning is visible here.

4. Customers Tab

Displays all registered customers with their membership level and accumulated loyalty points, with search by name or email. A form allows new customer registration, demonstrating a straightforward Create operation on Customers_046.

5. Orders Tab

Lists all orders with customer name, date, status, and total amount, sorted by most recent. Staff can update an order's status (e.g., from "Pending" to "Completed")

through a dedicated form. Setting an order's status to "Completed" fires the `trg_member_points_046` trigger, which awards the customer `FLOOR(total_amount)` loyalty points, updates their `total_points` balance in `Customers_046`, and inserts a record into `Points_Record_046`. A NOT EXISTS guard in the trigger ensures the same order cannot award points twice even if updated to "Completed" more than once.

Demonstration Flow Suggested for Screenshots

1. Log in as `manager1`, add a new product (Products tab).
2. Log in as `cashier1` (or continue), place an order for a customer, adding a product close to its reorder threshold — screenshot the Inventory tab before and after to show the stock drop and the new low-stock warning.
3. Update that order's status to "Completed" in the Orders tab — screenshot the Customers tab before and after to show `total_points` increasing, and query `Points_Record_046` to show the new row.

This flow exercises every table, both triggers, and all three views in a single continuous demo, and mirrors the actual operational sequence of a real store transaction from checkout through fulfillment.